Lab 7: Fuzzing for Input Validation Bugs

Visa Application module (L6_L7_52) — Hypothesis fuzz testing

Name: Arya Devang **SRN:** PES1UG24AM054

Objective

Use property-based fuzzing (Hypothesis) to find input-validation bugs in `visa_application.py`, then harden the three functions (`parse_application`, `processing_average`, `payable_fee`) so that no input can crash them.

Step 1: Install dependencies

Installed `hypothesis` and `pytest` with pip.

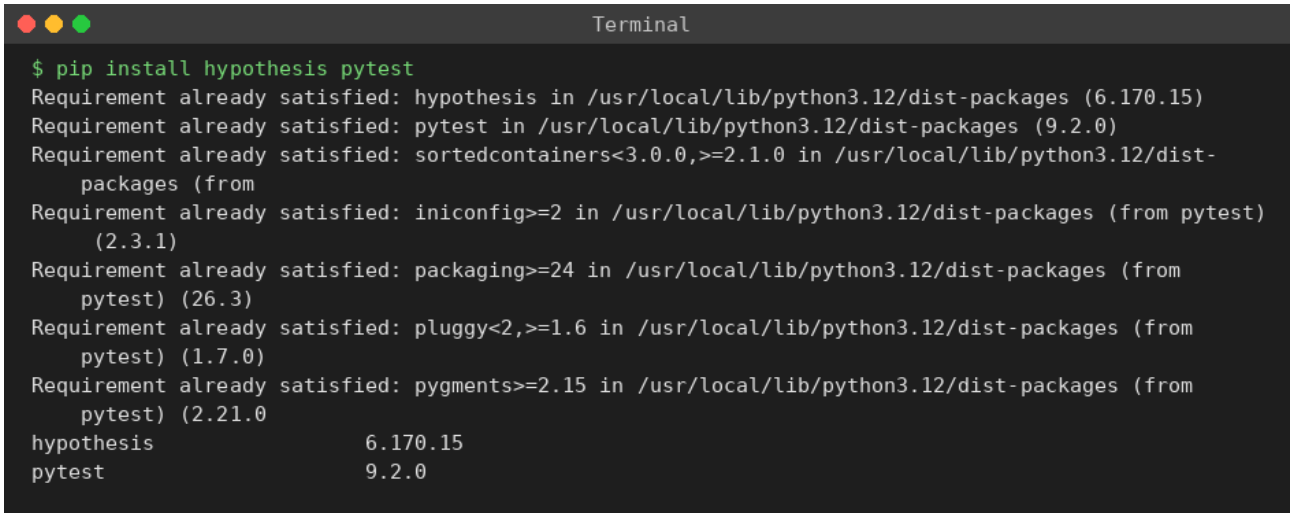


Figure 1: Dependencies installed (hypothesis 6.170.15, pytest 9.2.0)

Step 2: Run the buggy application

With well-formed input the original script works. With a malformed input (`tourist`, no colon) it crashes with an unhandled exception.

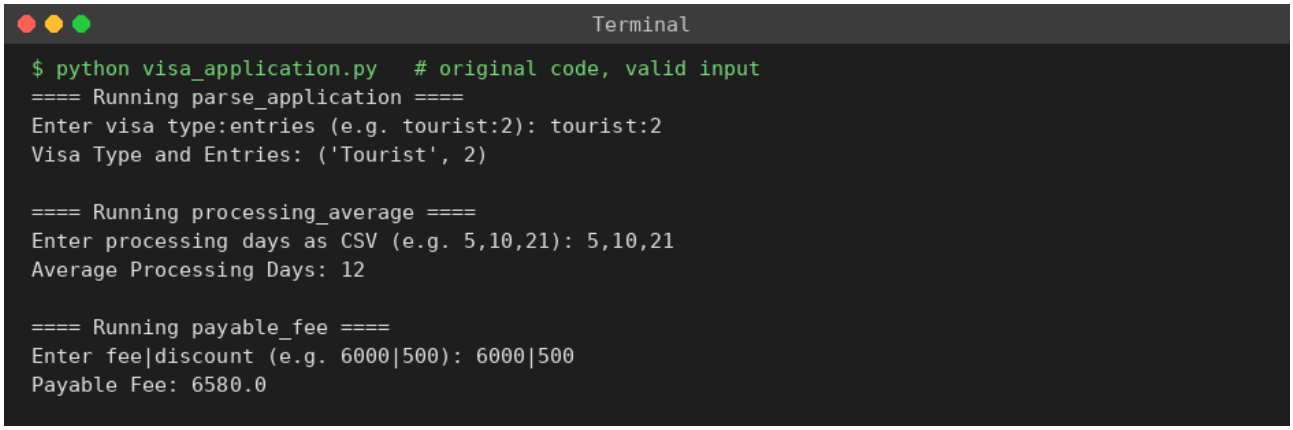


Figure 2a: Original code, valid input

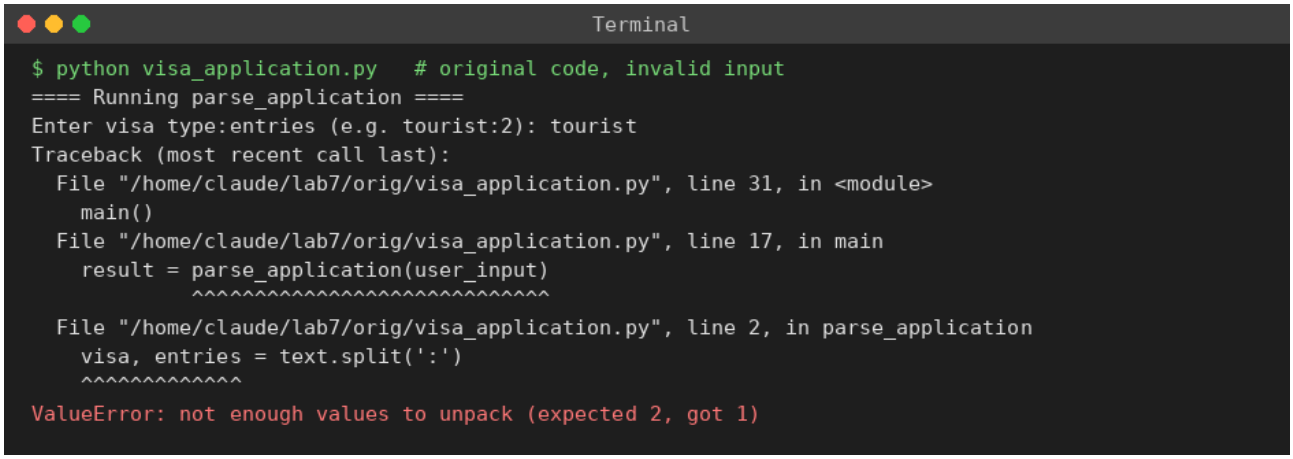


Figure 2b: Original code, malformed input → ValueError traceback

Step 3: Hypothesis tests

Each function gets a fuzz test using `st.text() | st.none()`. The property is: **the function must never raise an exception**. I also added three extra property tests that check correct results for valid input (so that 'fixing' a function by making it always return None would not pass).

```
from hypothesis import given, strategies as st
from visa_application import parse_application, processing_average, payable_fee


# ---------- Fuzz tests: the functions must never crash ----------

@given(st.text() | st.none())
def test_parse_application_no_crash(s):
    try:
        parse_application(s)
    except Exception as e:
        assert False, f"parse_application crashed with: {e!r}"


@given(st.text() | st.none())
def test_processing_average_no_crash(s):
    try:
        processing_average(s)
    except Exception as e:
        assert False, f"processing_average crashed with: {e!r}"


@given(st.text() | st.none())
def test_payable_fee_no_crash(s):
    try:
        payable_fee(s)
    except Exception as e:
        assert False, f"payable_fee crashed with: {e!r}"


# ---------- Extra property tests: valid input must give correct output ----------

@given(st.sampled_from(["tourist", "student", "work visa"]),
       st.integers(min_value=1, max_value=10**6))
def test_parse_application_valid(visa, entries):
    assert parse_application(f"{visa}:{entries}") == (visa.title(), entries)


@given(st.lists(st.integers(min_value=0, max_value=10**6), min_size=1))
def test_processing_average_valid(days):
    csv = ",".join(str(d) for d in days)
    assert processing_average(csv) == sum(days) // len(days)


@given(st.integers(min_value=0, max_value=10**6),
       st.integers(min_value=0, max_value=10**6))
def test_payable_fee_never_negative(fee, discount):
    result = payable_fee(f"{fee}|{discount}")
    assert result is not None and result >= 0
```

Step 4: Run the tests against the buggy code

All three fuzz tests fail (the three valid-input tests pass). Hypothesis shrinks each failure to a minimal input.

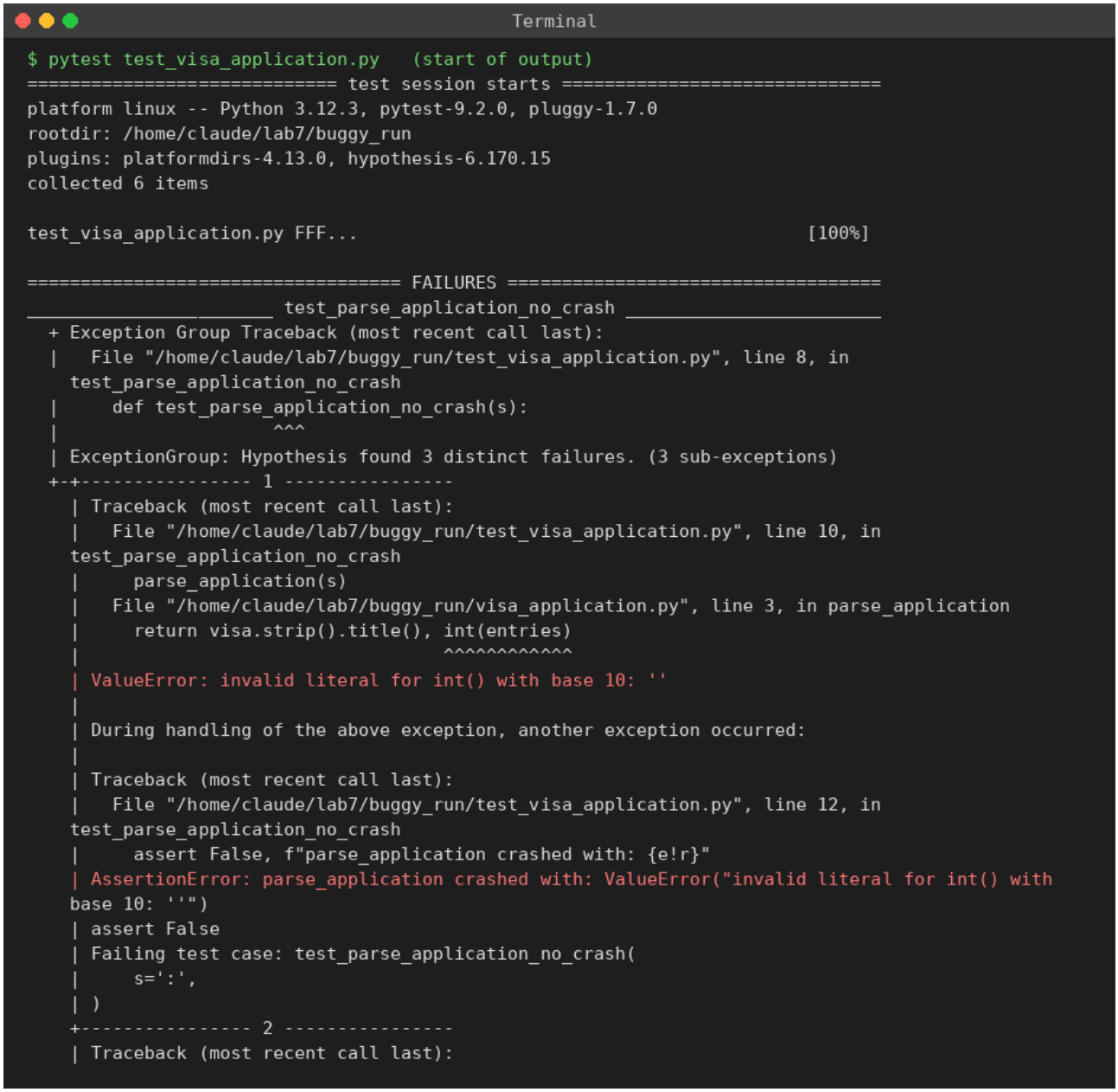


Figure 4a: pytest output (beginning)

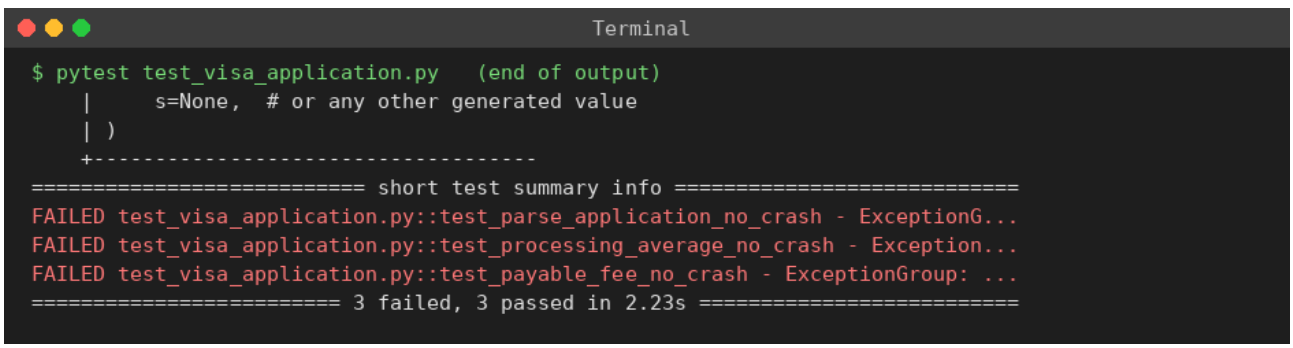


Figure 4b: pytest summary — 3 failed, 3 passed

Bugs found by fuzzing

Function	Minimal failing input	Exception	Root cause
parse_application	'' (empty string)	ValueError	split(':') gives 1 part; unpacking into 2 variables fails
parse_application	':'	ValueError	entries is '' so int('') fails
parse_application	None	AttributeError	None has no .split()
processing_average	''	ValueError	int('') fails on an empty element
processing_average	None	AttributeError	None has no .split()
payable_fee	'|'	ValueError	float('') fails on empty fee/discount
payable_fee	''	ValueError	split('|') gives 1 part; unpacking fails
payable_fee	None	AttributeError	None has no .split()

Other latent bugs fixed while hardening (not all are shown by the default fuzz run, but are reachable): `float('nan')`, `'inf'` and `'1e999'` parse successfully and give nan/inf fees; a negative fee, discount, entry count or processing day is accepted; `fee * 1.18` can overflow to inf; entries of 0 are accepted.

Step 5: Fix the application

Changes made to `visa_application.py`:

• Every function first checks `isinstance(x, str)`, which handles None and other types.
• `split()` results are length-checked before unpacking (exactly 2 parts for `:` and `|`).
• `int()`/`float()` conversions are wrapped in try/except ValueError (covers empty and non-numeric text, and very long digit strings).
• Business rules enforced: non-empty visa type, entries ≥ 1, non-negative days, fee and discount; nan/inf and overflow rejected via `math.isfinite`.
• Invalid input returns `None` and `main()` prints a friendly message instead of a traceback.

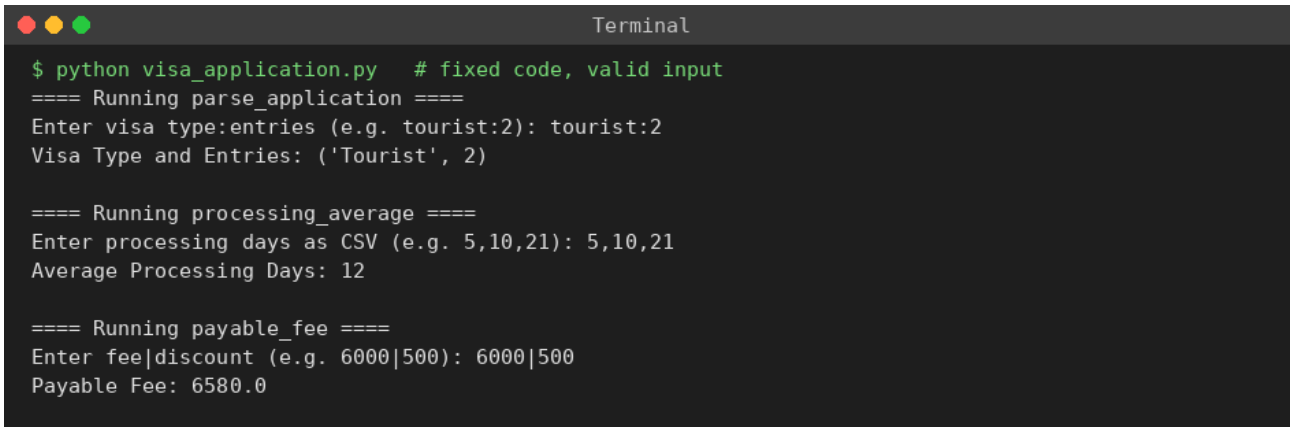


Figure 5a: Fixed code, valid input (same results as before)

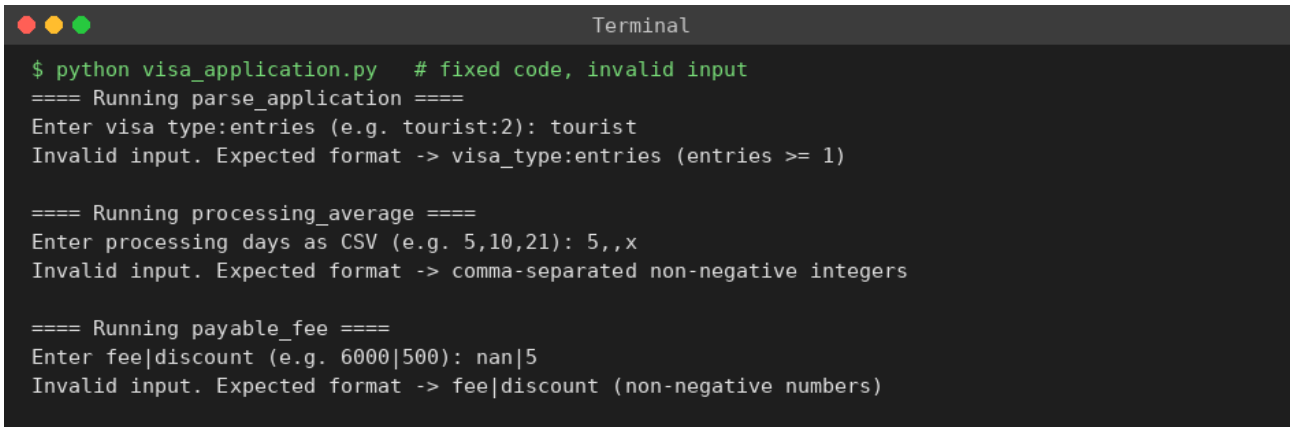


Figure 5b: Fixed code, invalid input → clean error messages, no crash

Step 6: Re-run the tests against the fixed code

All 6 tests pass, including the three fuzz tests.

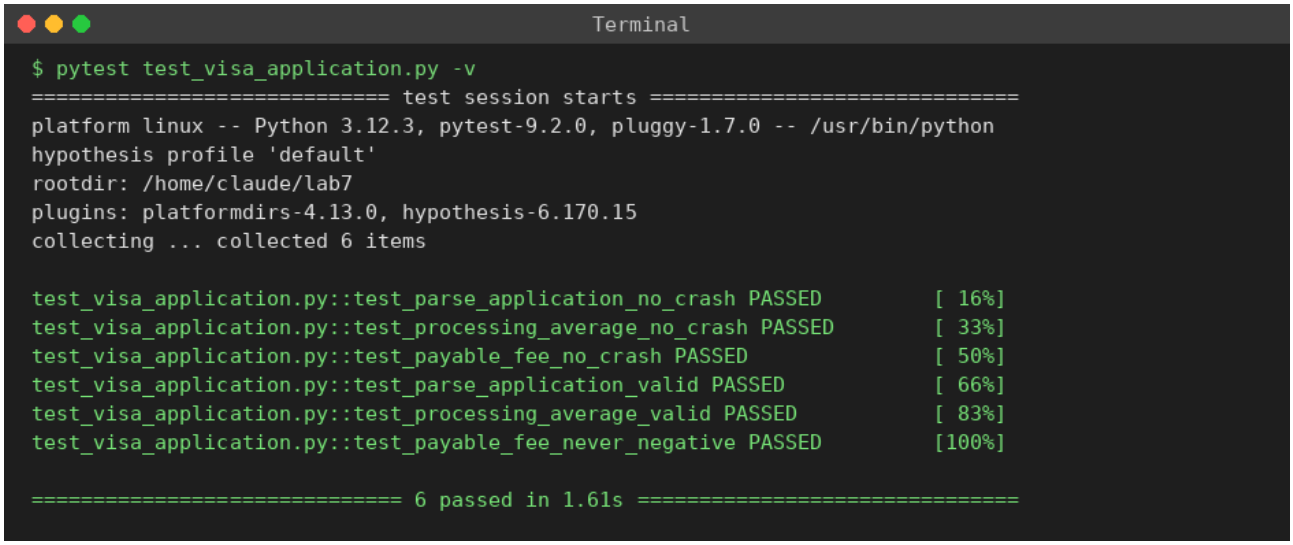


Figure 6: pytest on fixed code — 6 passed

Conclusion

Fuzzing exposed that the original functions assumed well-formed input: None, empty strings and missing delimiters all raised unhandled exceptions. By validating type, structure, and value ranges before converting, and returning a safe sentinel (None) for invalid input, the functions are now robust against arbitrary text. The extra property tests confirm that valid input still produces correct results.

Appendix: fixed visa_application.py

```
import math


def parse_application(text):
    """Parse 'visa_type:entries' (e.g. 'tourist:2') into ('Tourist', 2).

    Returns None for any invalid input instead of raising.
    """
    if not isinstance(text, str):
        return None

    parts = text.split(':')
    if len(parts) != 2:
        return None

    visa, entries = parts[0].strip(), parts[1].strip()
    if not visa:
        return None

    try:
        entries = int(entries)
    except ValueError:  # non-numeric, empty, or absurdly long digit string
        return None

    if entries < 1:  # a visa must allow at least one entry
        return None

    return visa.title(), entries


def processing_average(days_csv):
    """Average (floor) of comma-separated processing days, e.g. '5,10,21' -> 12.

    Returns None for any invalid input instead of raising.
    """
    if not isinstance(days_csv, str):
        return None

    values = []
    for part in days_csv.split(','):
        try:
            days = int(part.strip())
        except ValueError:  # empty element, letters, huge digit string
            return None
        if days < 0:  # negative processing time makes no sense
            return None
        values.append(days)

    if not values:
        return None

    return sum(values) // len(values)


def payable_fee(text):
    """Compute 'fee|discount' -> fee * 1.18 (GST) - discount, never below 0.

    Returns None for any invalid input instead of raising.
    """
    if not isinstance(text, str):
        return None

    parts = text.split('|')
    if len(parts) != 2:
        return None

    try:
        fee = float(parts[0])
        discount = float(parts[1])
    except ValueError:
        return None

    # reject nan / inf (float() happily parses 'nan', 'inf', '1e999')
    if not (math.isfinite(fee) and math.isfinite(discount)):
```

```
        return None

    if fee < 0 or discount < 0:
        return None

    total = fee * 1.18 - discount
    if not math.isfinite(total):  # fee * 1.18 can overflow to inf
        return None

    return max(total, 0)


def main():
    print("==== Running parse_application ====")
    user_input = input("Enter visa type:entries (e.g. tourist:2): ")
    result = parse_application(user_input)
    if result is None:
        print("Invalid input. Expected format -> visa_type:entries (entries >= 1)")
    else:
        print("Visa Type and Entries:", result)

    print("\n==== Running processing_average ====")
    user_input = input("Enter processing days as CSV (e.g. 5,10,21): ")
    result = processing_average(user_input)
    if result is None:
        print("Invalid input. Expected format -> comma-separated non-negative integers")
    else:
        print("Average Processing Days:", result)

    print("\n==== Running payable_fee ====")
    user_input = input("Enter fee|discount (e.g. 6000|500): ")
    result = payable_fee(user_input)
    if result is None:
        print("Invalid input. Expected format -> fee|discount (non-negative numbers)")
    else:
        print("Payable Fee:", result)


if __name__ == "__main__":
    main()
```